

Assignment Report

Title: **Compiler Principles**

The Fifth Theory Homework

Name	林宏宇
Student ID	23320093
School	计算机学院
Major	计算机科学与技术
Instructor	李文军

[ABSTRACT]

After learning the lecture 5 of compiler principles[1], we got the homework at the end of the lecture.

There are some requirements for the homework:

- The file should be named as **“学号 _ 姓名 _ 第 i 次理论作业”**, where i is the number of the homework.
- The file should be in docx or **pdf** format.
- All the homework should be sent to the TA’s email: **406648896@qq.com**
- The deadline for the homework is **24:00 on Sunday of the week** when the homework is assigned.

Keywords: Compiler Principles

Contents

1 Exercise 5.1 1
1.1 Problem 1
1.2 Solution 1

2 Exercise 5.2** 2
2.1 Problem 2
2.2 Solution 3

References 5

1 Exercise 5.1

1.1 Problem

Given the following grammar:

$$S \to (L) \mid a$$

$$L \to L, S \mid S$$

Tasks:

1) Construct an LL(1) parsing table for the grammar. (Note: you must eliminate the left recursion first.)
2) Draw the detailed process of the parsing of the sentence "(a, (a, a))", follow the style in the previous slides.

1.2 Solution

1.2.1 Step 1: Eliminate left recursion

The nonterminal L is left-recursive. We rewrite L as:

$$S \to (L) \mid a$$

$$L \to SL'$$

$$L' \to , SL' \mid \epsilon$$

1.2.2 Step 2: FIRST and FOLLOW sets

- FIRST$(S) = \{(, a\}$
- FIRST(L) = FIRST$(S) = \{(, a\}$
- FIRST$(L') = \{, , \epsilon\}$
- FOLLOW$(S) = \{, ,), \$\}$ (Note: S is followed by L' which starts with ',' or ends L, and L is followed by ')')

- FOLLOW(L) = {)}
- FOLLOW(L') = FOLLOW(L) = {)}

1.2.3 Step 3: LL(1) Parsing Table

Nonterminal	(	)	a	,	$
S	$S \to (L)$		$S \to a$		
L	$L \to SL'$		$L \to SL'$		
L'		$L' \to \epsilon$		$L' \to, SL'$	

1.2.4 Step 4: Parsing process for "(a, (a, a))"

Stack	Input	Action	Output
$S	(a, (a, a))	derive	$S \to (L)$
$) L (	(a, (a, a))	match (	
$) L	a, (a, a))	derive	$L \to SL'$
$) L' S	a, (a, a))	derive	$S \to a$
$) L' a	a, (a, a))	match a	
$) L'	, (a, a))	derive	$L' \to, SL'$
$) L' S ,	, (a, a))	match ,	
$) L' S	(a, a))	derive	$S \to (L)$
$) L') L (	(a, a))	match (	
$) L') L	a, a))	derive	$L \to SL'$
$) L') L' S	a, a))	derive	$S \to a$
$) L') L' a	a, a))	match a	
$) L') L'	, a))	derive	$L' \to, SL'$
$) L') L' S ,	, a))	match ,	
$) L') L' S	a))	derive	$S \to a$
$) L') L' a	a))	match a	
$) L') L'	))	derive	$L' \to \epsilon$
$) L')	))	match)	
$) L'	)	derive	$L' \to \epsilon$
$)	)	match)	

2 Exercise 5.2**

2.1 Problem

Given the following grammar:

$$A \to B \mid BC$$

$$B \to aB \mid \epsilon$$

$$C \to ab$$

Tasks:

1) Left factor the grammar.
2) After left factoring, is the grammar an LL(1) grammar? or is it an LL(k) grammar? and why? (Note: you may try the input string “ab”.)

2.2 Solution

2.2.1 Step 1: Left factoring

The production $A \to B \mid BC$ has a common prefix B. We factor it as:

$$A \to BA'$$

$$A' \to C \mid \epsilon$$

$$B \to aB \mid \epsilon$$

$$C \to ab$$

2.2.2 Step 2: Analysis

Grammar after left-factoring:

$$A \to BA', \quad A' \to C \mid \epsilon, \quad B \to aB \mid \epsilon, \quad C \to ab.$$

Proof:

Not LL(1):

Consider the input string “ab”. On seeing the first symbol ‘a’ we cannot decide with only one-symbol lookahead whether to use $B \to aB$ or $B \to \epsilon$: if $B \to \epsilon$ then $A \Rightarrow BA' \Rightarrow C \Rightarrow ab$ matches “ab”, but if $B \to aB$ the sentential form will contain an extra ‘a’ and cannot produce “ab”. Thus a FIRST-set conflict exists, so the grammar is not LL(1).

LL(2):

With two-symbol lookahead the choice is unambiguous: if the next two symbols are

"ab" choose $B \to \epsilon$ (so $A' \to C \to ab$); if they are "aa" choose $B \to aB$ to consume the extra 'a'. Therefore a predictive parser with $k = 2$ has no conflicts, so the grammar is LL(2).

Conclusion: the grammar is not LL(1) but is LL(2).

References

[1] JUN LI W. Lecture05. principles of compiler construction[Z]. 2026.